

第12章

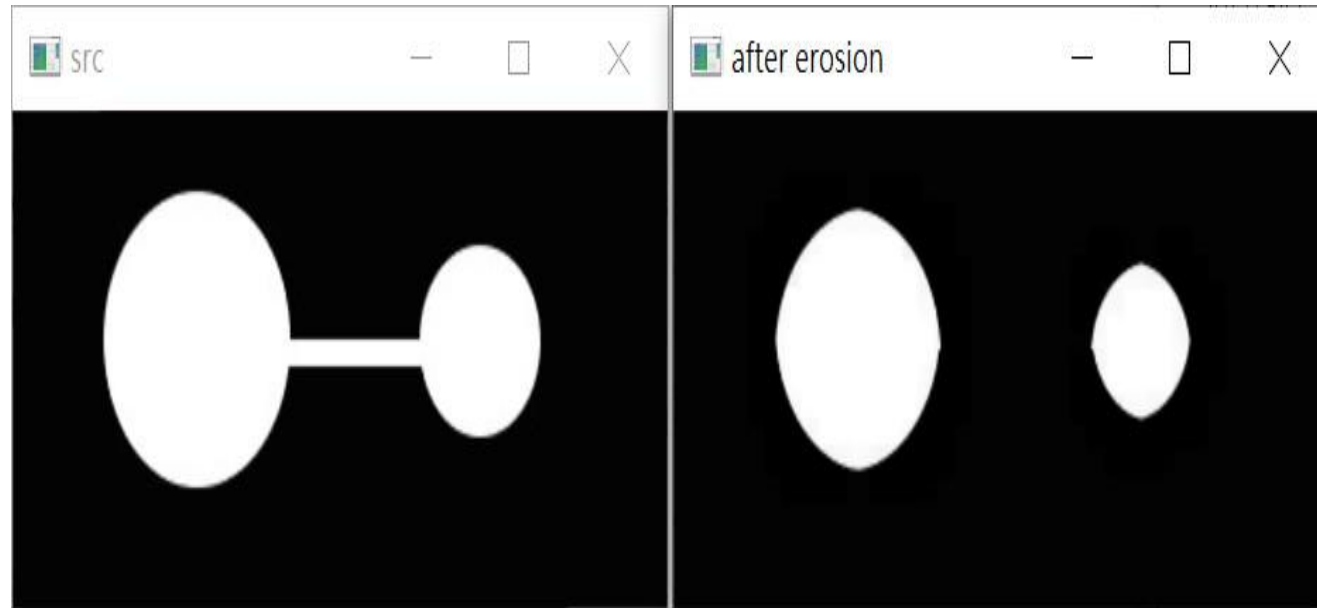
數學形態學

- 在電腦視覺領域有時又叫影像形態學。影像分析學科建立在格論(lattice)和拓撲學(topology)的基礎上。
- 此學科基本的理論運算包含：
 - 腐蝕(Erosion)
 - 膨脹(Dilation)
 - 開運算(Opening)
 - 閉運算(Closing)
 - 形態學梯度(morphological gradient)
 - 禮帽運算(tophat)
 - 黑帽運算(blackhat)

12-1：腐蝕(Erosion)

• 12-1-1：理論基礎

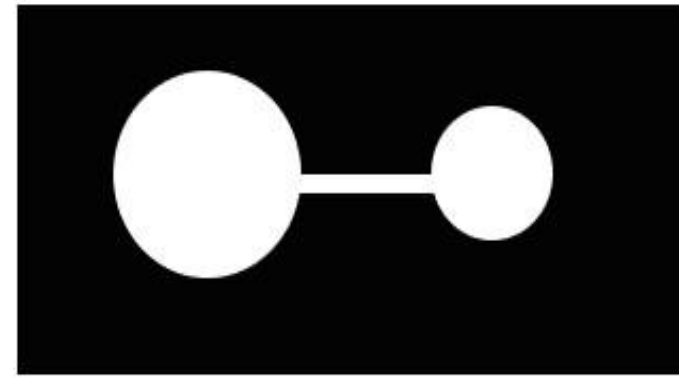
- 腐蝕是讓影像**沿著邊緣**向內收縮的操作，其目的有三：
 - 刪除影像中的雜點，相當於刪除影像噪音
 - 細化影像，刪除毛邊
 - 可以分割影像



- 腐蝕操作一般常用於二進制的影像。

- **腐蝕操作原理：**

假設原始影像前景顏色是白色(像素值是1)，背景顏色是黑色(像素值是0)，我們需要**建立一個內核(kernel)**或是稱核，內核的中心素稱核心



原始影像

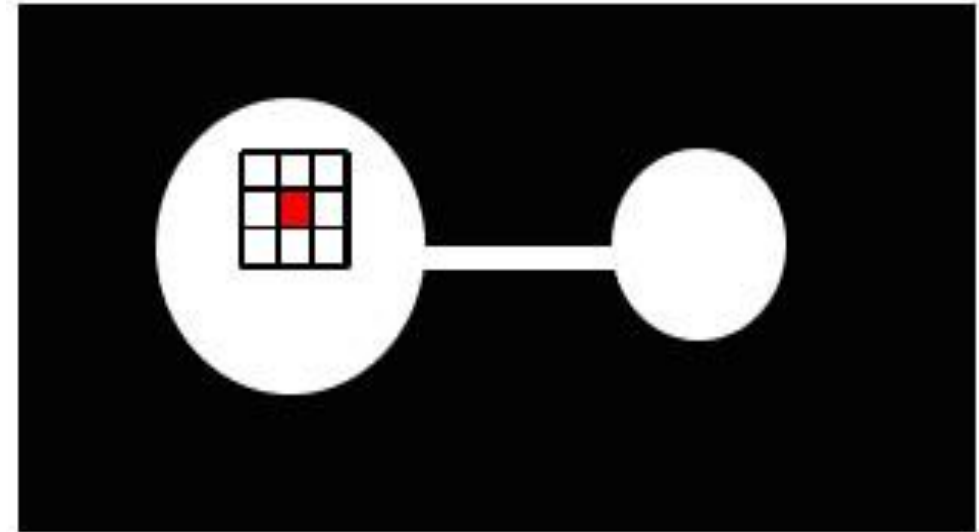


內核(kernel)

- 腐蝕操作是讓內核遍歷影像所有的像素點，然後由內核的**核心**決定目前影像像素點的像素值，這時會發生2種情況。

- **情況1：**

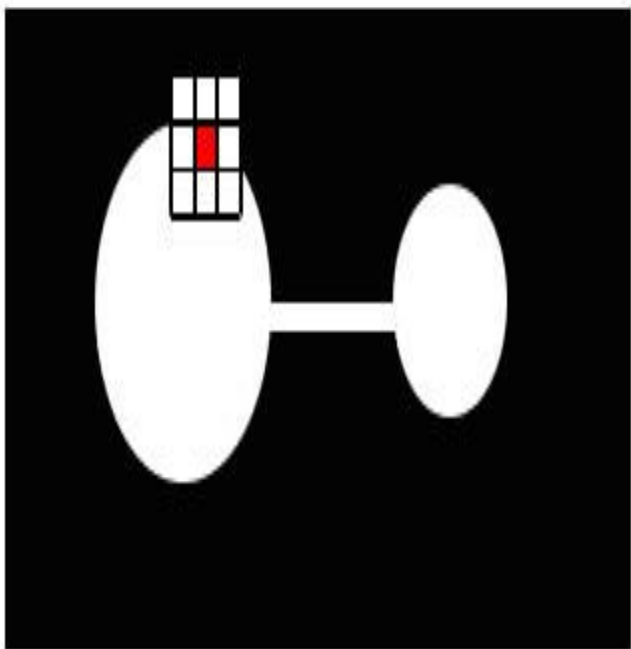
- 如果內核完全在原始影像中，則內核的**核心**所在的像素點設定為**前景顏色**，也就是白色，可以參考情況1圖。



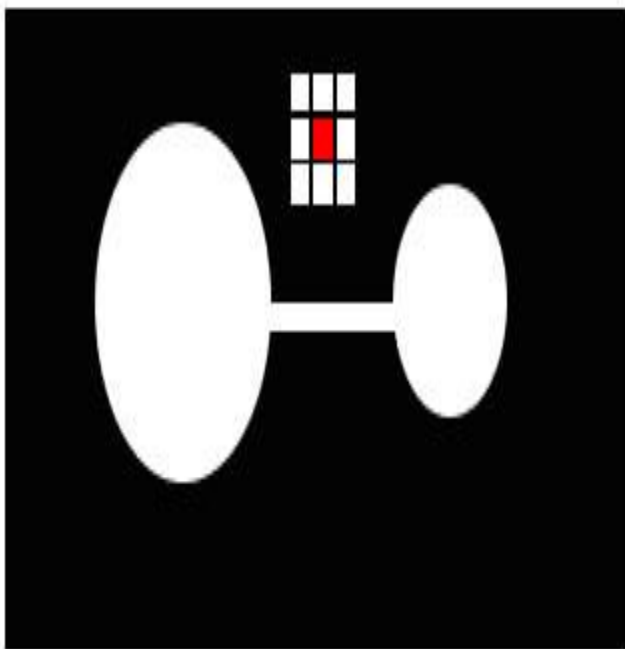
情況 1

- 情況2：

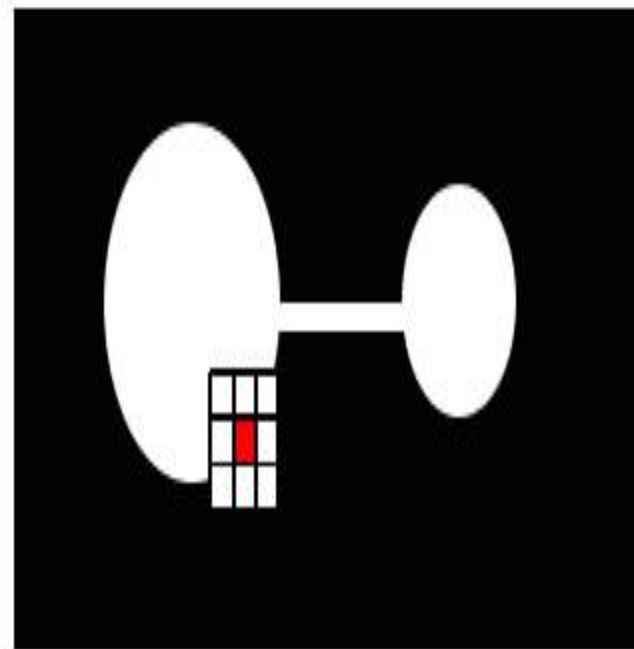
- 如果內核**完全不在**原始影像中，或是**部分**在原始影像中，則將目前內核的核心所在的像素點設定為背景色，也就是黑色，可以參考下圖。



內核部分在原始影像中



內核完全不在原始影像中



內核部分在原始影像中

12-1-2：腐蝕函數

- 腐蝕概念以`erode( )`函數來操作
- `dst = cv2.erode(src, kernel, anchor, iterations, boderType, borderValue)`
- Src: 來源影像或原始影像
- Kernel: 腐蝕操作的內核, 可自行定義也可用`getStructuringElement()`產生 (12-9節)
- Anchor: 設定內核錨點位置，也就是設定核心位置，預設值(-1,-1), 表示使用中心點點錨點
- Iterations: 腐蝕操作的重覆次數, 預設為1
- boderType: 邊界樣式，建議使用預設值
- borderValue：邊界值，建議用預設值

程式實例：使用陣列當作原始影像，設定3 x 3的內核進行腐蝕操作

```
import cv2
import numpy as np

src = np.zeros((7,7),np.uint8)
src[1:6,1:6] = 1          # 建立前景影像
kernel = np.ones((3,3),np.uint8) # 建立內核
dst = cv2.erode(src, kernel)    # 腐蝕操作
print(f"src = \n {src}")
print(f"kernel = \n {kernel}")
print(f"Erosion = \n {dst}")
```

```
===== RESTART: D:/OpenCV_Python/ch12/ch12_1.py =====
src =
[[0 0 0 0 0 0 0]
 [0 1 1 1 1 1 0]
 [0 1 1 1 1 1 0]
 [0 1 1 1 1 1 0]
 [0 1 1 1 1 1 0]
 [0 1 1 1 1 1 0]
 [0 0 0 0 0 0 0]]
kernel =
[[1 1 1]
 [1 1 1]
 [1 1 1]]
Erosion =
[[0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0]
 [0 0 1 1 1 0 0]
 [0 0 1 1 1 0 0]
 [0 0 1 1 1 0 0]
 [0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0]]
```

當內核核心在此框內任一位置, 內核一定在影像內

所以最後這些像素值是1

0	0	0	0	0	0	0
0	1	1	1	1	1	0
0	1	1	1	1	1	0
0	1	1	1	1	1	0
0	1	1	1	1	1	0
0	1	1	1	1	1	0
0	0	0	0	0	0	0

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

紅色框內部指影像範圍

紫色框指內核, 內核有在影像內部則設為影像顏色**1**, 其餘不在影像內部或是部份在影像之中則設為**0**

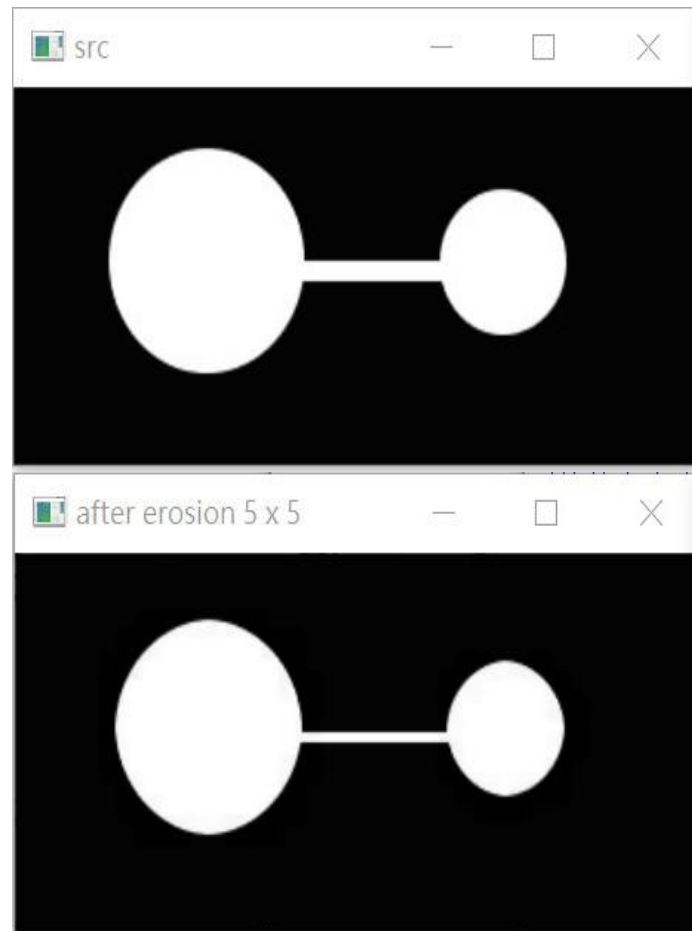
- 程式實例：使用5 x 5當作內核，然後針對bw.jpg執行腐蝕操作，最後列出執行結果。

```
import cv2
import numpy as np

src = cv2.imread("bw.jpg")
kernel = np.ones((5, 5), np.uint8)      # 建立5x5內核
dst1 = cv2.erode(src, kernel)           # 腐蝕操作

cv2.imshow("src", src)
cv2.imshow("after erosion 5 x 5", dst1)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



- 程式實例：使用彩色影像 [whilster.jpg](#) 重新設計 `ch12_2.py`，將腐蝕功能應用在彩色影像，可以看到深色調部分會增加，白色部分會減少。

- `import cv2`

- `import numpy as np`

- `src = cv2.imread("whilster.jpg")`

- `kernel = np.ones((3,3),np.uint8)` # 建立3x3內核

- `dst1 = cv2.erode(src, kernel)` # 腐蝕操作

- `kerne2 = np.ones((5,5),np.uint8)` # 建立5x5內核

- `dst2 = cv2.erode(src, kerne2)` # 腐蝕操作

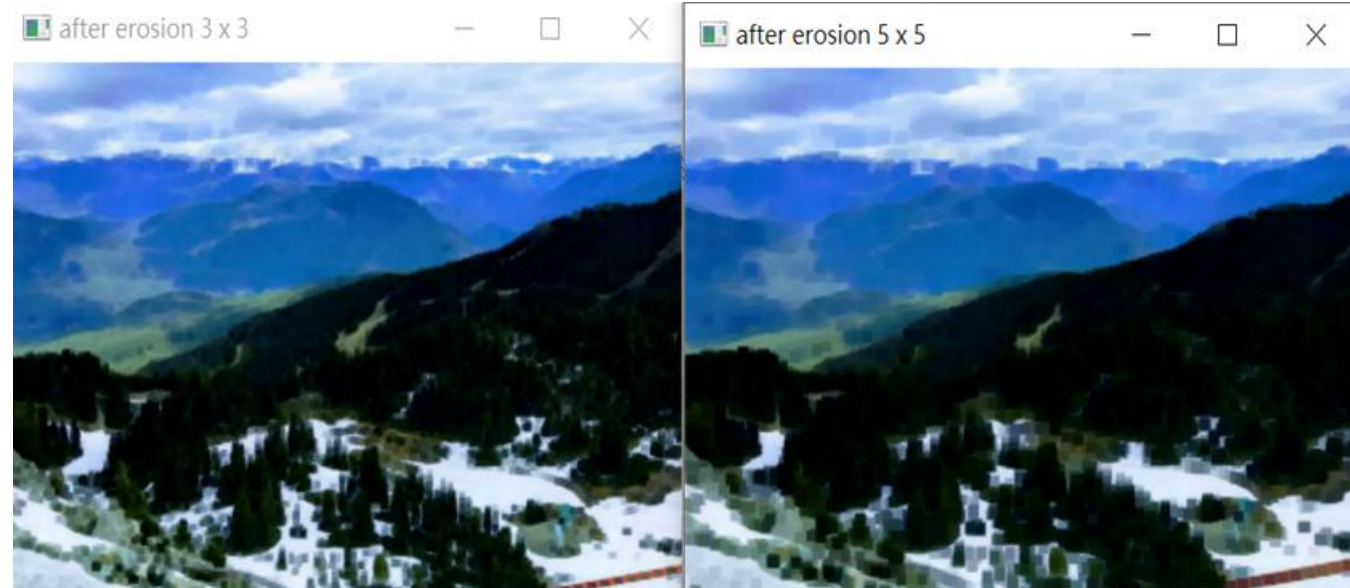
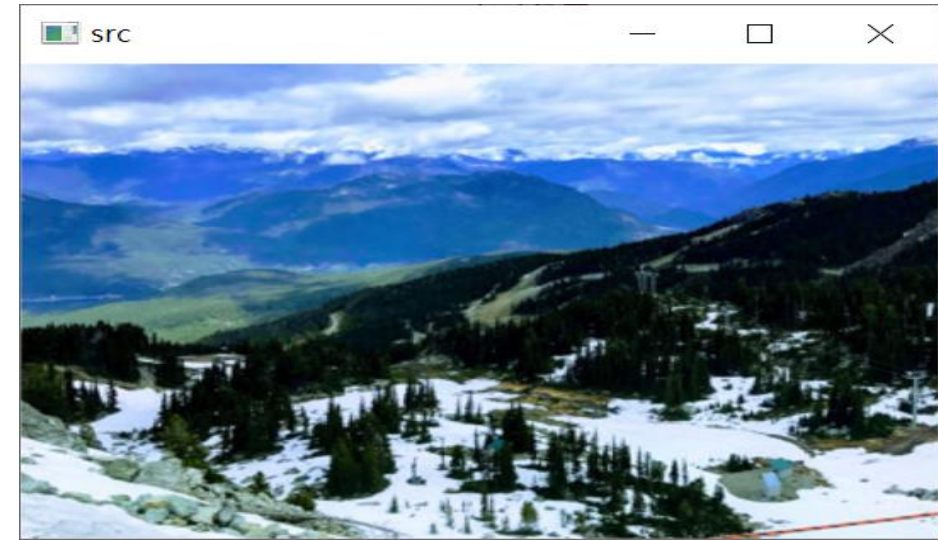
- `cv2.imshow("src",src)`

- `cv2.imshow("after erosion 3 x 3",dst1)`

- `cv2.imshow("after erosion 5 x 5",dst2)`

- `cv2.waitKey(0)`

- `cv2.destroyAllWindows()`



12-9：核函數

- kernel = cv2.getStructuringElement(shape, ksize, anchor)

具名常數	說明
MORPH_RECT	所有元素值皆是 1
MORPH_ELLIPSE	橢圓形結構是 1
MORPH_CROSS	十字形元素位置是 1

```
import cv2
import numpy as np
kernel = cv2.getStructuringElement(cv2.MORPH_RECT,(5,5))
print(f"MORPH_RECT \n {kernel}")
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE,(5,5))
print(f"MORPH_ELLIPSE \n {kernel}")
kernel = cv2.getStructuringElement(cv2.MORPH_CROSS,(5,5))
print(f"MORPH_CROSS \n {kernel}")
```

```
===== R
MORPH_RECT
[[1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]]
MORPH_ELLIPSE
[[0 0 1 0 0]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [0 0 1 0 0]]
MORPH_CROSS
[[0 0 1 0 0]
 [0 0 1 0 0]
 [1 1 1 1 1]
 [0 0 1 0 0]
 [0 0 1 0 0]]
```

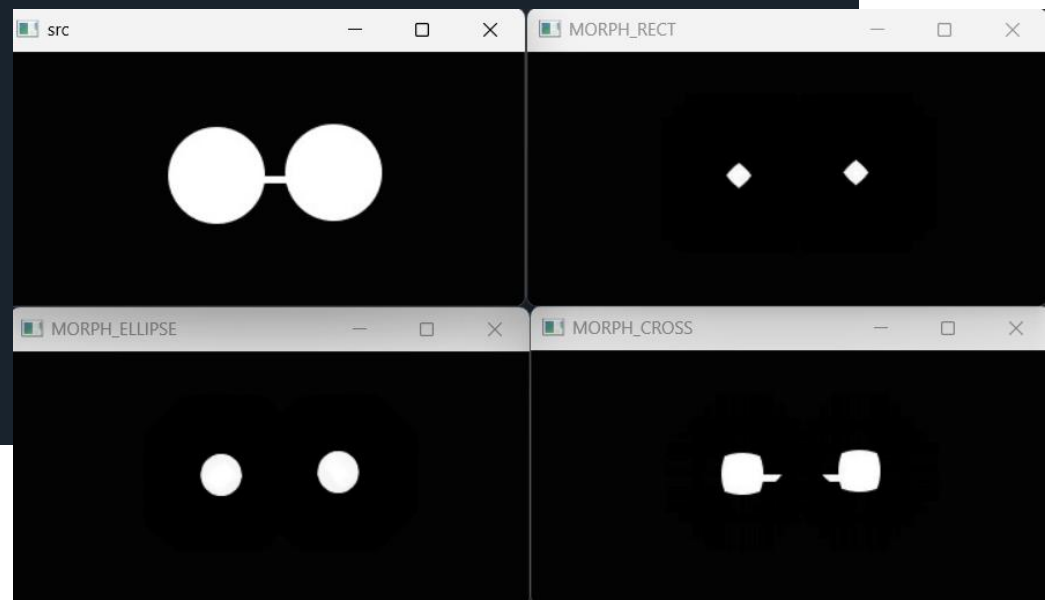
- 使用`getStructuringElement()`分別建立`MORPH_RECT`、`MORPH_ELLIPSE`和`MORPH_CROSS`不同外形內核(kernel), 進行`erode` (腐蝕) 操作

```
# ch12_22.py
import cv2
import numpy as np

src = cv2.imread("bw_circle.jpg")
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (39,39))
dst1 = cv2.erode(src, kernel)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (39,39))
dst2 = cv2.erode(src, kernel)
kernel = cv2.getStructuringElement(cv2.MORPH_CROSS, (39,39))
dst3 = cv2.erode(src, kernel)

cv2.imshow("src", src)
cv2.imshow("MORPH_RECT", dst1)
cv2.imshow("MORPH_ELLIPSE", dst2)
cv2.imshow("MORPH_CROSS", dst3)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



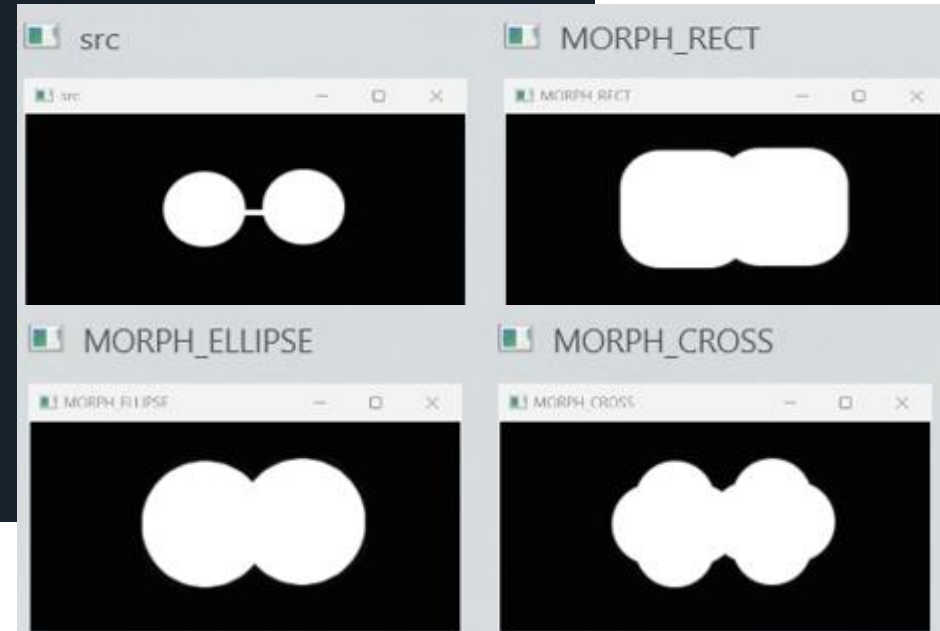
- 使用`getStructuringElement()`分別建立MORPH_RECT、MORPH_ELLIPSE和MORPH_CROSS不同外形內核(kernel), 進行dilate (膨脹) 操作

```
import cv2
import numpy as np

src = cv2.imread("bw_circle.jpg")
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (39, 39))
dst1 = cv2.dilate(src, kernel)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (39, 39))
dst2 = cv2.dilate(src, kernel)
kernel = cv2.getStructuringElement(cv2.MORPH_CROSS, (39, 39))
dst3 = cv2.dilate(src, kernel)

cv2.imshow("src", src)
cv2.imshow("MORPH_RECT", dst1)
cv2.imshow("MORPH_ELLIPSE", dst2)
cv2.imshow("MORPH_CROSS", dst3)

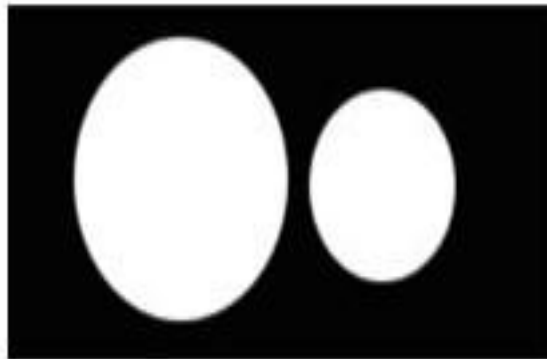
cv2.waitKey(0)
cv2.destroyAllWindows()
```



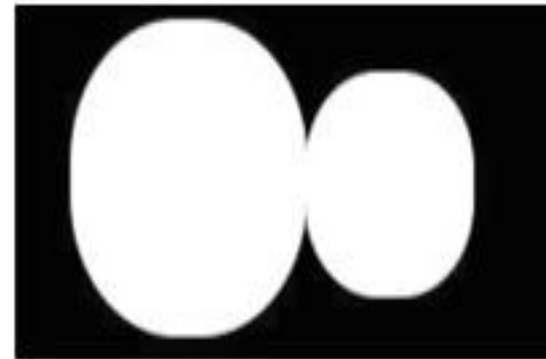
12-2：膨脹(Dilation)

• 12-2-1：理論基礎

- 膨脹是影像**沿著邊緣向外擴張**的操作，等同腐蝕反向操作。反覆擴張有時可以讓兩個分開影像連接



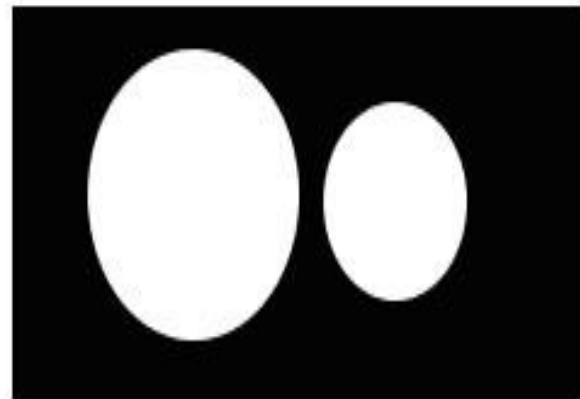
原始影像



膨脹後結果

• 膨脹操作原理：

- 假設原始影像前景顏色是白色(像素值是1)，背景顏色是黑色(像素值是0)，我們需要**建立一個內核(kernel)**或是稱核，內核的中心素稱核心
- 膨脹操作是讓**內核**遍歷影像所有的像素點，然後由內核的**核心**決定目前影像像素點的像素值，這時會發生2種情況。



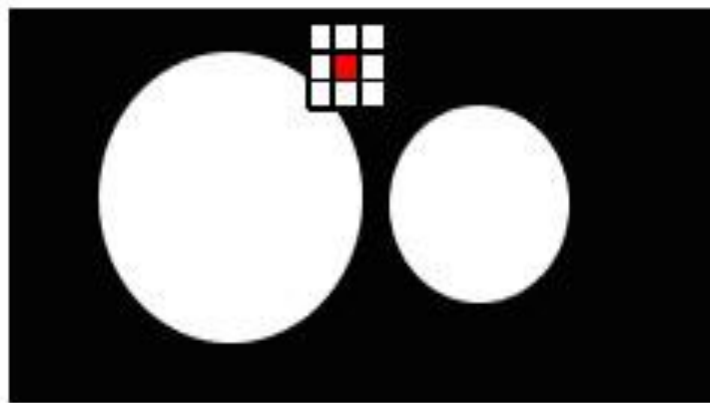
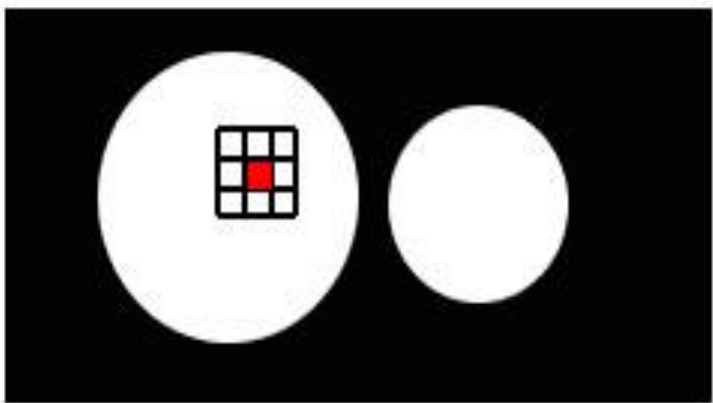
原始影像



內核(kernel)

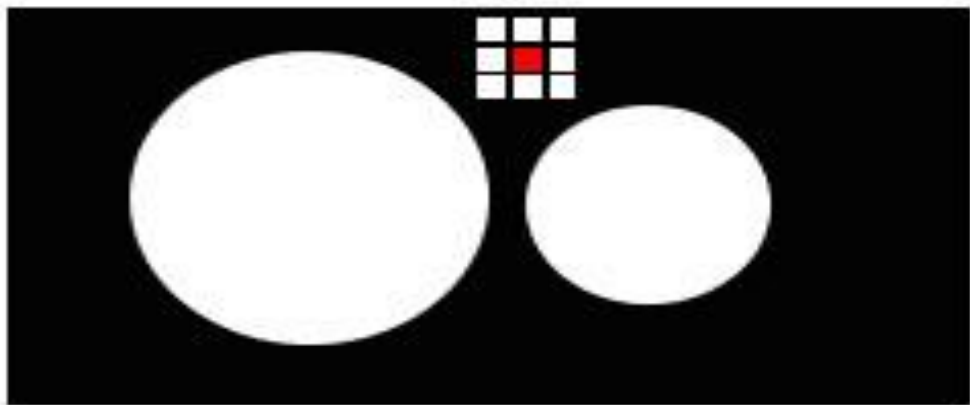
- 情況1：

- 如果內核全部或部分在原始影像中，則內核的核心目前所在的像素點是設定為前景顏色，也就是白色，可以參考下圖。



- 情況2：

- 如果內核完全不在原始影像中，則將目前內核的核心所在的像素點設定為背景色，也就是黑色，可以參考下圖。

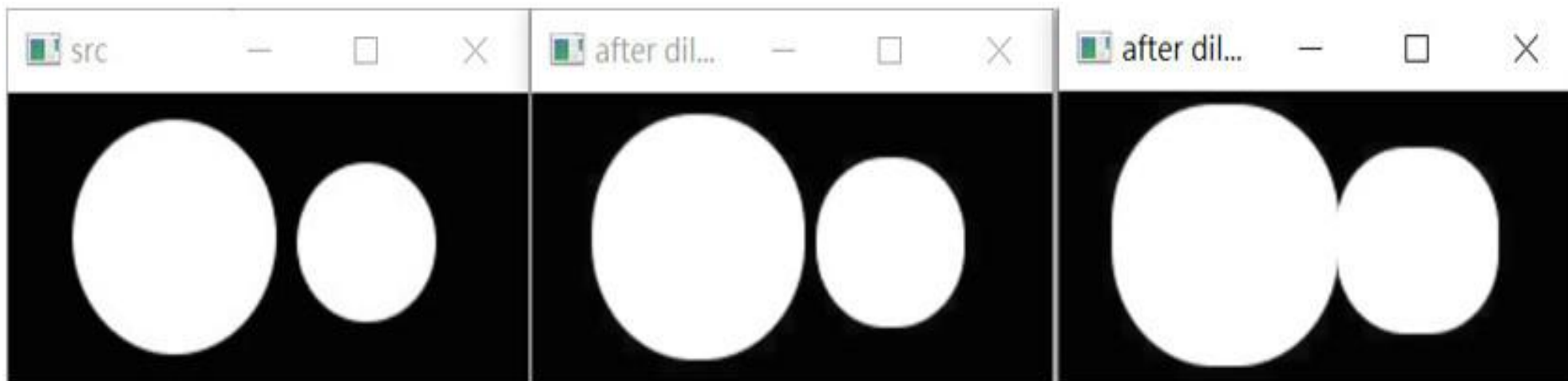


12-2-2：膨脹函數dilate()

- `dst = cv2.dilate(src, kernel, anchor, iterations, boderType, borderValue)`
- 程式實例：使用陣列當作原始影像，設定3 x 3的內核，了解膨脹操作。

```
src =  
[[0 0 0 0 0 0 0]  
 [0 0 0 0 0 0 0]  
 [0 0 1 1 1 0 0]  
 [0 0 1 1 1 0 0]  
 [0 0 1 1 1 0 0]  
 [0 0 0 0 0 0 0]  
 [0 0 0 0 0 0 0]]  
  
kernel =  
[[1 1 1]  
 [1 1 1]  
 [1 1 1]]  
  
Dilation =  
[[0 0 0 0 0 0 0]  
 [0 1 1 1 1 1 0]  
 [0 1 1 1 1 1 0]  
 [0 1 1 1 1 1 0]  
 [0 1 1 1 1 1 0]  
 [0 1 1 1 1 1 0]  
 [0 0 0 0 0 0 0]]
```


- 程式實例：使用 5×5 和 11×11 當作內核，然後針對bw_dilate.jpg執行膨脹操作，最後列出執行結果。

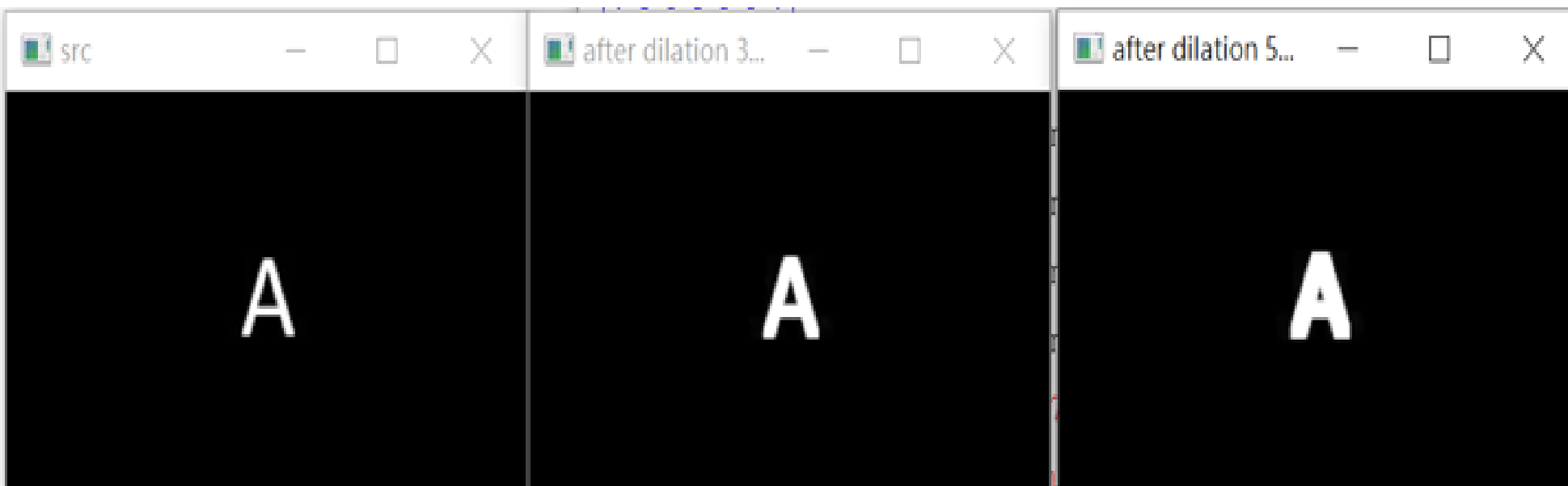


原始影像

核心是 5×5

核心是 11×11

- 程式實例練習：對A.jpg執行膨脹操作，設定核心是3 x 3和5 x 5，並列出結果。

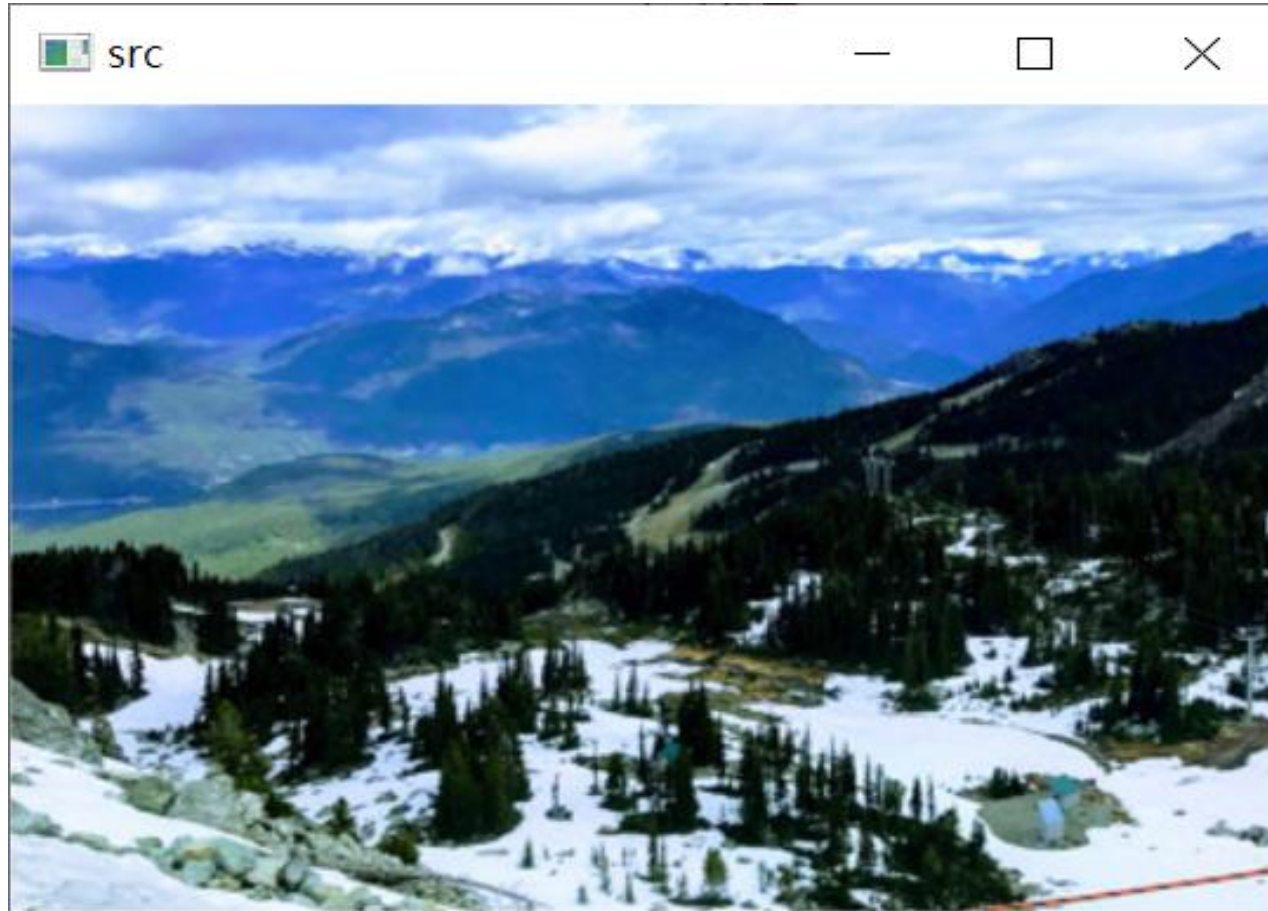


原始影像

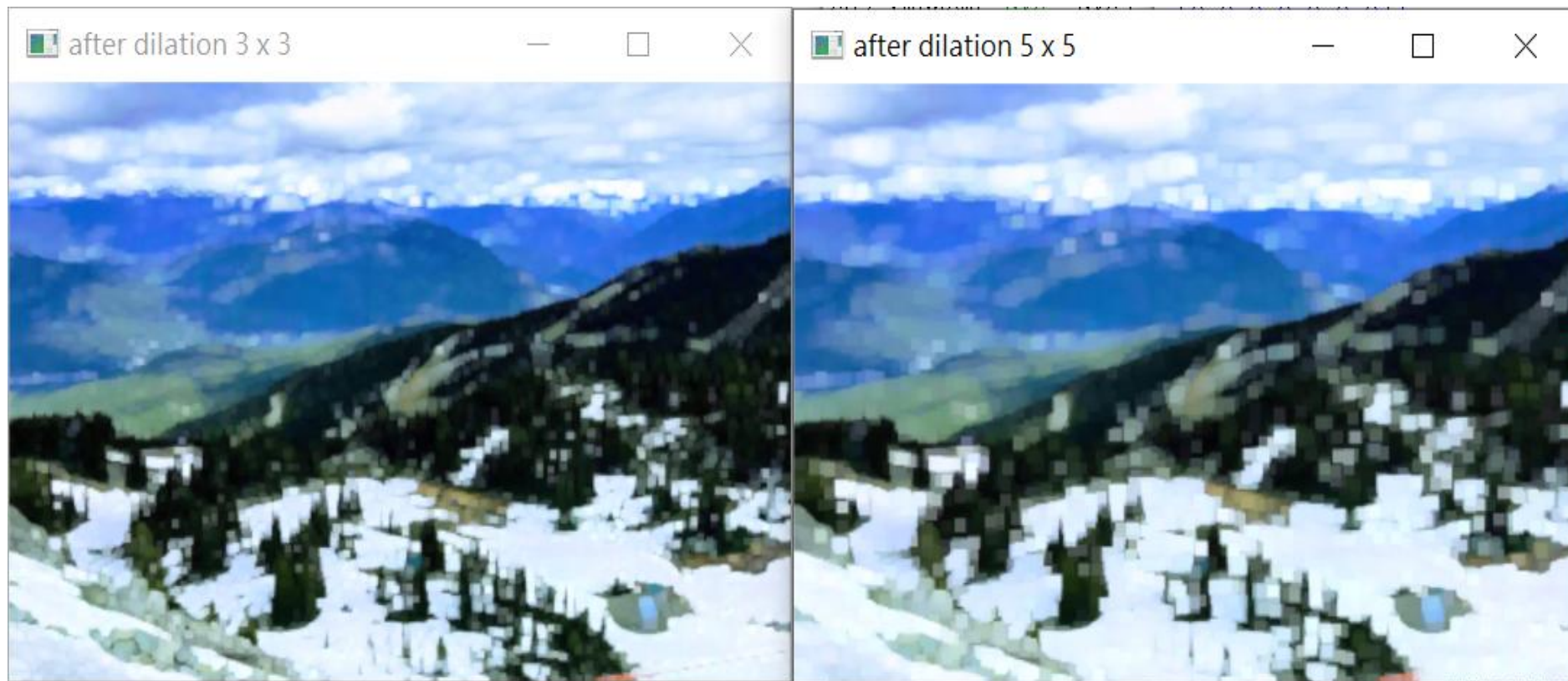
核心是3 x 3

核心是5 x 5

- 程式實例ch12_8.py：使用膨脹操作重新設計ch12_4.py。



- 下列左邊是 3×3 內核與右邊是 5×5 內核，膨脹操作的結果。



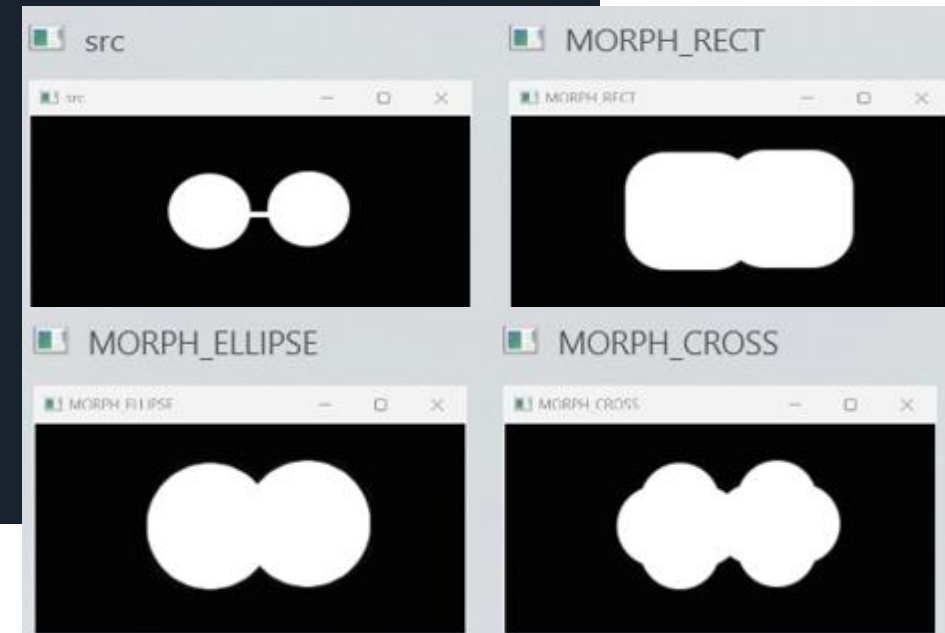
- 使用`getStructuringElement()`分別建立MORPH_RECT、MORPH_ELLIPSE和MORPH_CROSS不同外形內核(kernel), 進行dilate (膨脹) 操作

```
import cv2
import numpy as np

src = cv2.imread("bw_circle.jpg")
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (39, 39))
dst1 = cv2.dilate(src, kernel)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (39, 39))
dst2 = cv2.dilate(src, kernel)
kernel = cv2.getStructuringElement(cv2.MORPH_CROSS, (39, 39))
dst3 = cv2.dilate(src, kernel)

cv2.imshow("src", src)
cv2.imshow("MORPH_RECT", dst1)
cv2.imshow("MORPH_ELLIPSE", dst2)
cv2.imshow("MORPH_CROSS", dst3)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



- 一，腐蝕和膨脹
- 腐蝕和膨脹是最基本的形態學操作，腐蝕和膨脹都是針對白色部分（高亮部分）而言的。
- **膨脹**就是使影像中高亮部分擴張，效果圖擁有比原圖更大的高亮區域（是求**區域性最大值**的操作）
- **腐蝕**是原圖中的高亮區域被蠶食，效果圖擁有比原圖更小的亮區域（是求**區域性最小值**的操作）
- 膨脹與腐蝕能實現多種多樣的功能，主要如下：
 - 消除噪聲
 - 腐蝕分割(isolate)出獨立的影像元素，膨脹在影像中連線(join)相鄰的元素。
 - 尋找影像中的明顯的極大值區域或極小值區域
 - 求出影像的梯度

練習題

- 圖像tw_noise.jpg

12-3：OpenCV應用在數學形態學的通用函數

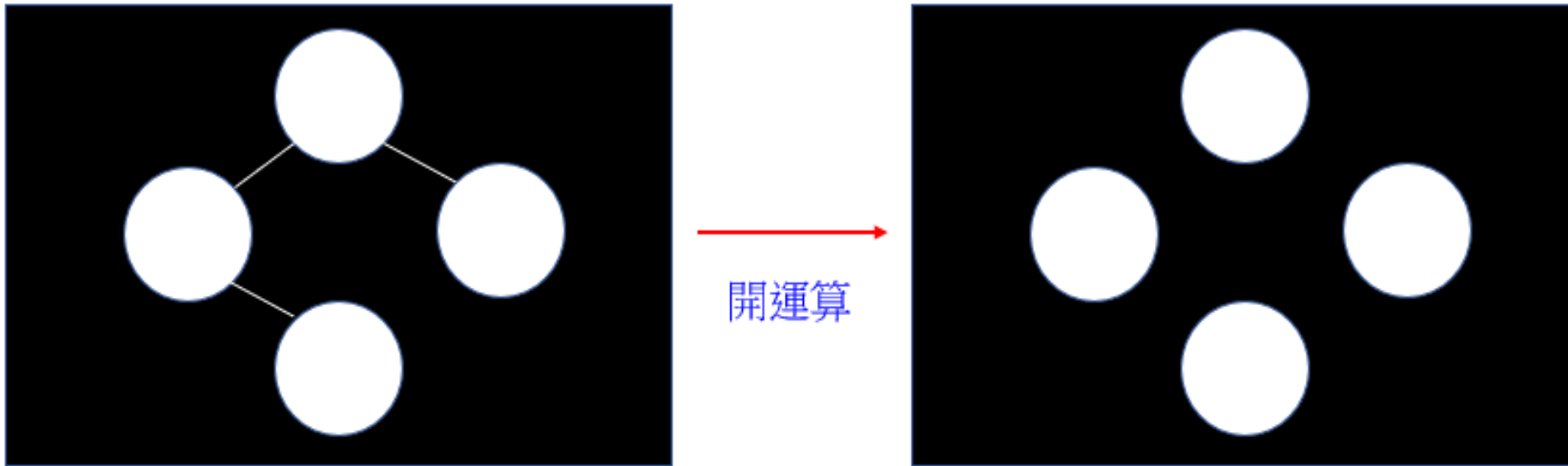
- 基礎運算：腐蝕(Erosion)和膨脹(Dilation)
- 擴充運算：開運算(Opening)、閉運算(Closing)、形態學梯度(morphological gradient)、禮帽運算(tophat)、黑帽運算(blackhat)等。OpenCV對這些擴充的運算提供了通用函數morphologyEx()，此函數語法如下：
- `dst = cv2.morphologyEx(src, op, kernel, anchor, iteration, boderType, boderValue)`
- Kernel: 操作定義的內核(可getStructuringElement產生)
- Anchor: 設定內核錨點位置, 預設(-1,-1)
- Iterations: 指定形態操作迭代次數, 預設1
- broderType: 邊界樣式, 用預設即可
- broderValue: 邊界值, 用預設即可

Op(操作方式):

具名常數	說明	說明
MORPH_ERODE	腐蝕	erode()
MORPH_DILATE	膨脹	dilate()
MORPH_OPEN	開運算	dilate(erode())
MORPH_CLOSE	閉運算	erode(dilate())
MORPH_GRADIENT	形態學梯度	dilate() - erode()
MORPH_TOPHAT	頂帽運算	src - open()
MORPH_BLACKHAT	黑帽運算	close() - src
MORPH_HITMISS	擊中擊不中	intersection(erode(src), erode(src1))

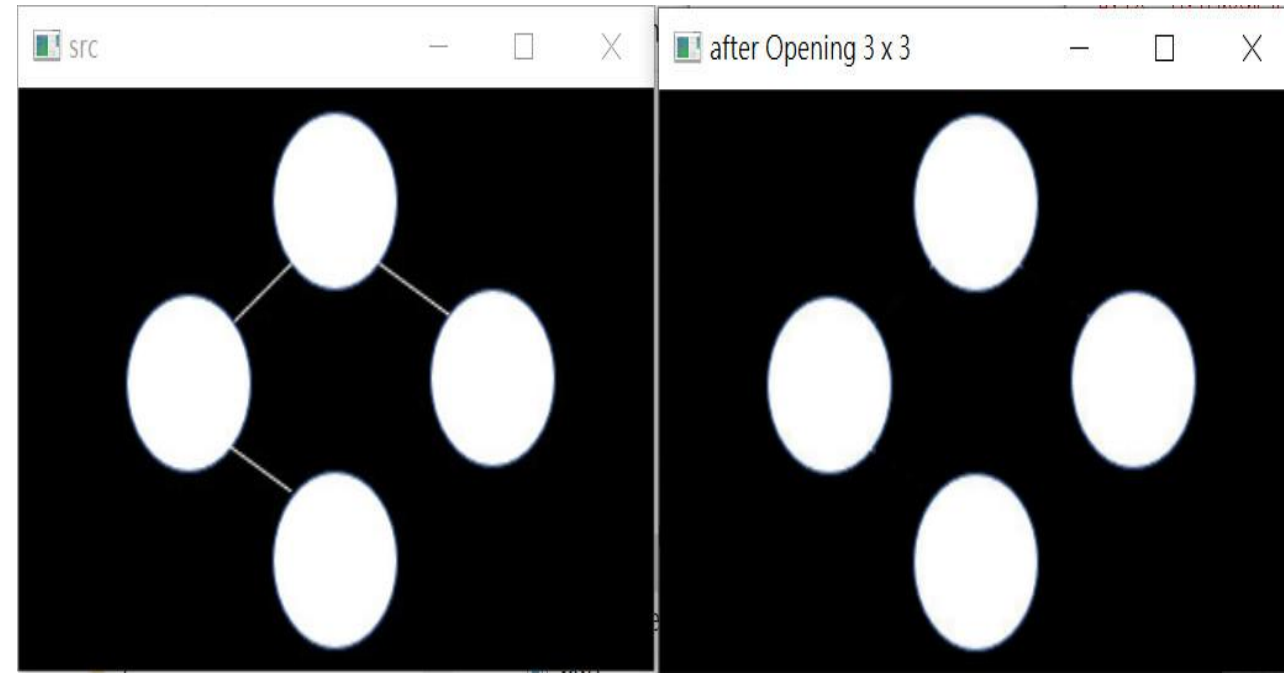
12-4：開運算(Opening)

- 所謂的開運算(Opening)就是將影像先做腐蝕(erosion)操作，然後再做膨脹(dilation)操作，這個操作主要可以刪除影像噪音。
- 開運算可以刪除噪音。適度的使用也可以完成更多的工作，例如：二元樹經過開運算後可以得到下圖右邊的結果，再使用檢驗輪廓，即可獲得二元樹的節點數量

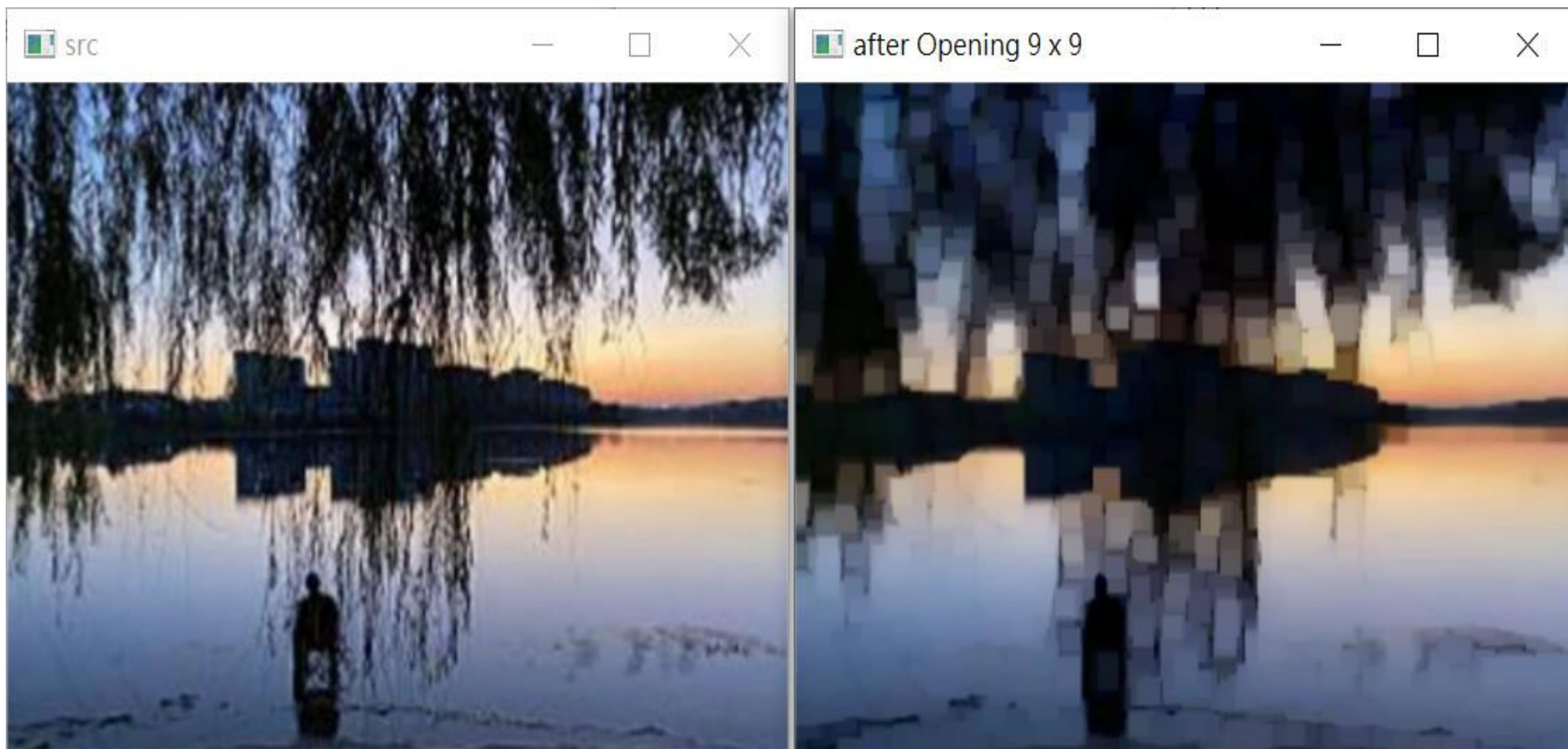


- 程式實例：建立3 x 3內核，使用開運算對二元樹操作。

- `import cv2`
- `import numpy as np`
- `src = cv2.imread("btree.jpg")`
- `kernel = np.ones((3,3),np.uint8)` # 建立3x3內核
- `dst = cv2.morphologyEx(src,cv2.MORPH_OPEN,kernel)` # 開運算
- `cv2.imshow("src",src)`
- `cv2.imshow("after Opening 3 x 3",dst)`
- `cv2.waitKey(0)`
- `cv2.destroyAllWindows()`

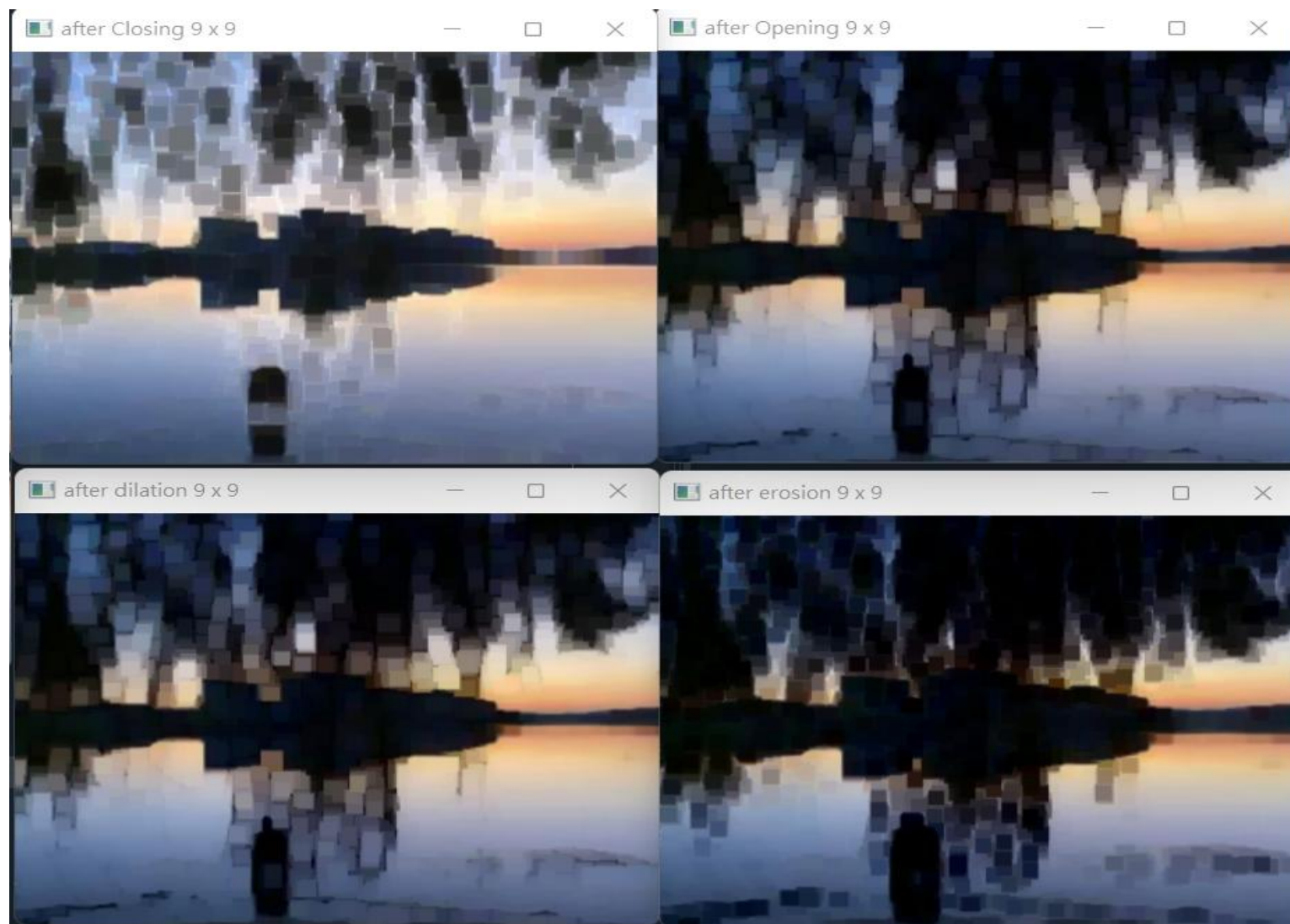


- 程式實例ch12_10.py：建立9 x 9內核，觀察開運算對彩色影像的影響。



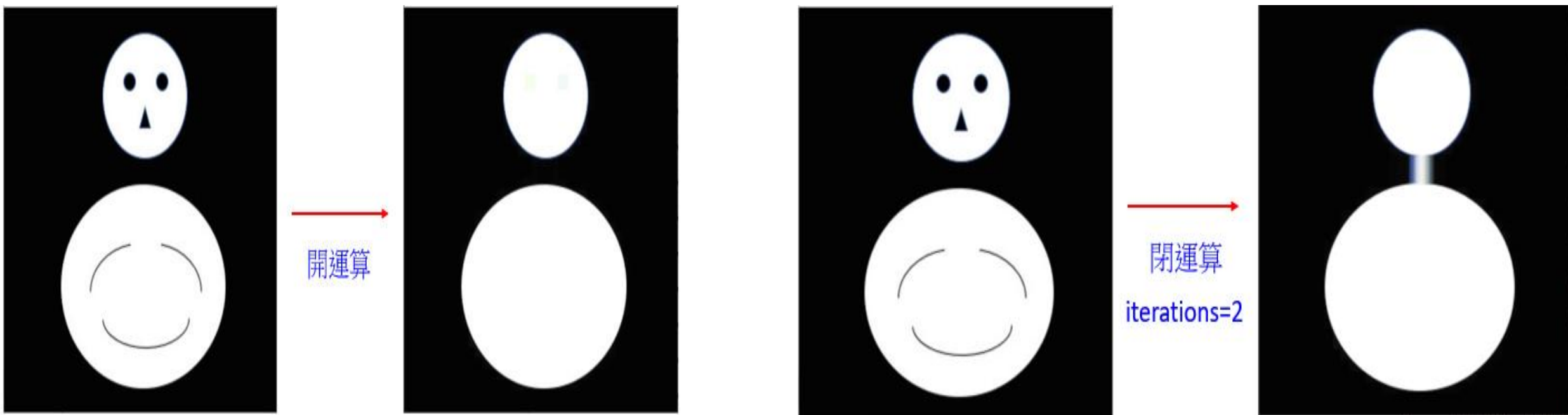
腐蝕，膨脹，開運算，閉運算的處理比較

影像: night.jpg



12-5：閉運算(Closing)

- 所謂的閉運算(Closing)就是將影像先做膨脹(dilation)操作，然後再做腐蝕(erosion)操作，這個操作主要可以刪除影像前景的噪音或是其他小細節。

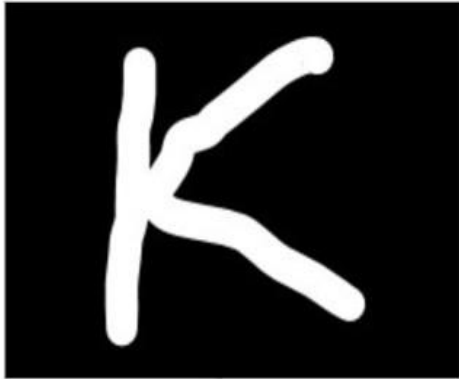


內核大小:11x11

- 程式練習2：影像snowman1.jpg

12-6：形態學梯度(Morphological gradient)

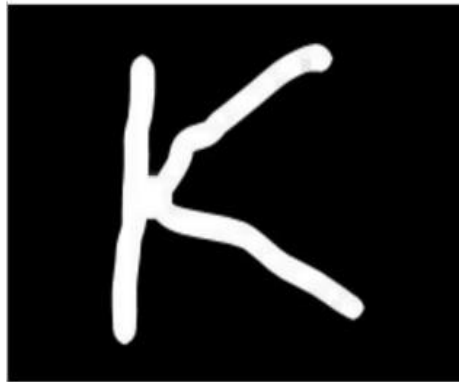
- 形態學梯度是一種獲影像邊緣的演算法，其觀念為：
- 膨脹的影像 **減去** 腐蝕的影像 = 影像的邊緣。



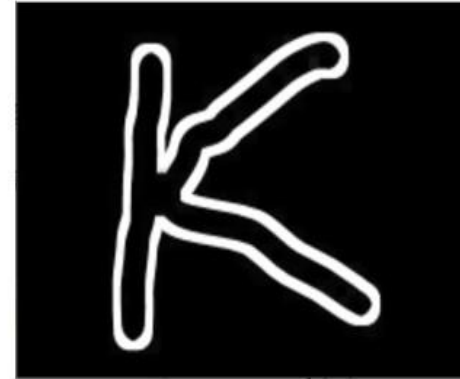
原始影像



膨脹影像

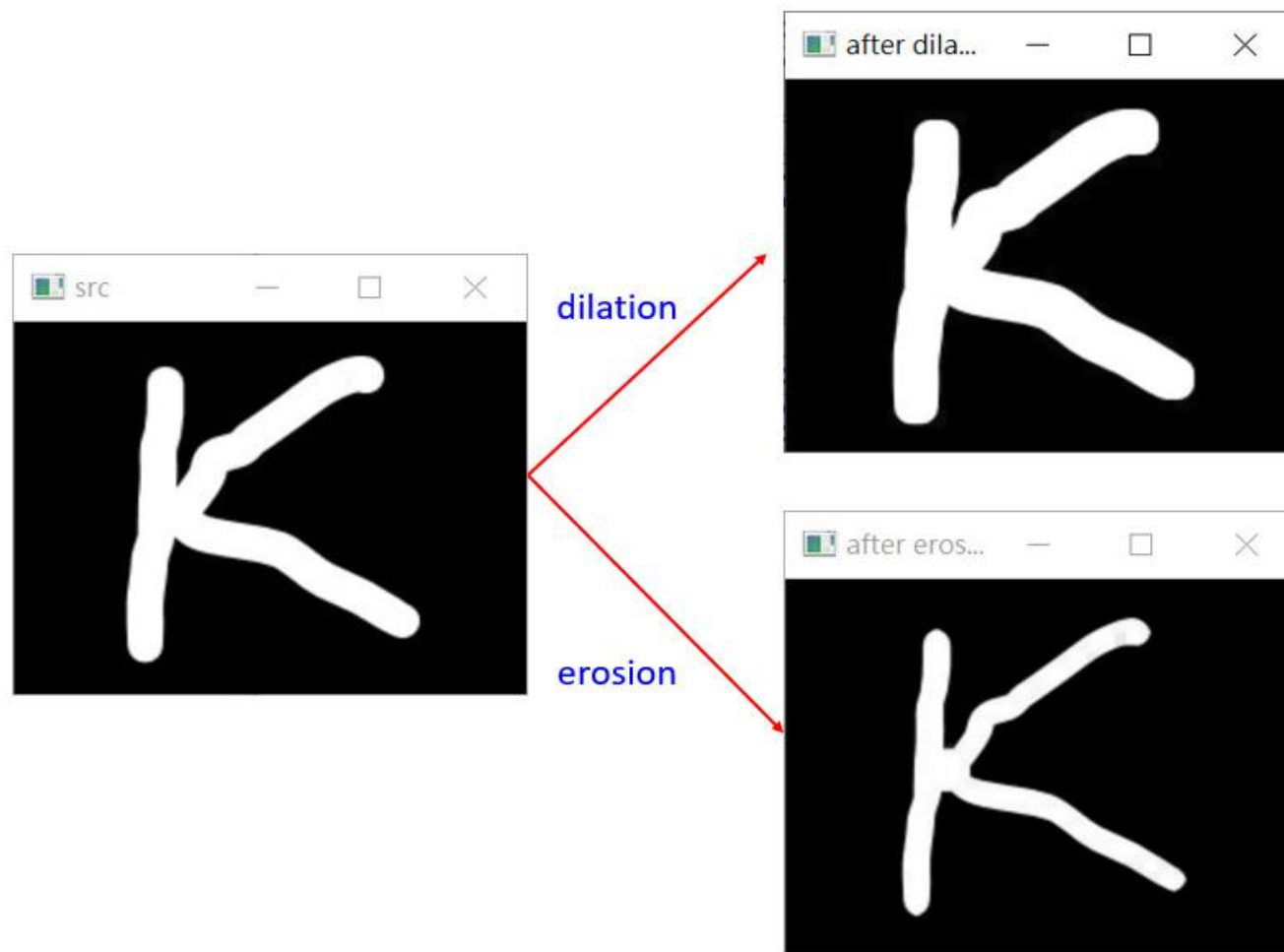


腐蝕影像

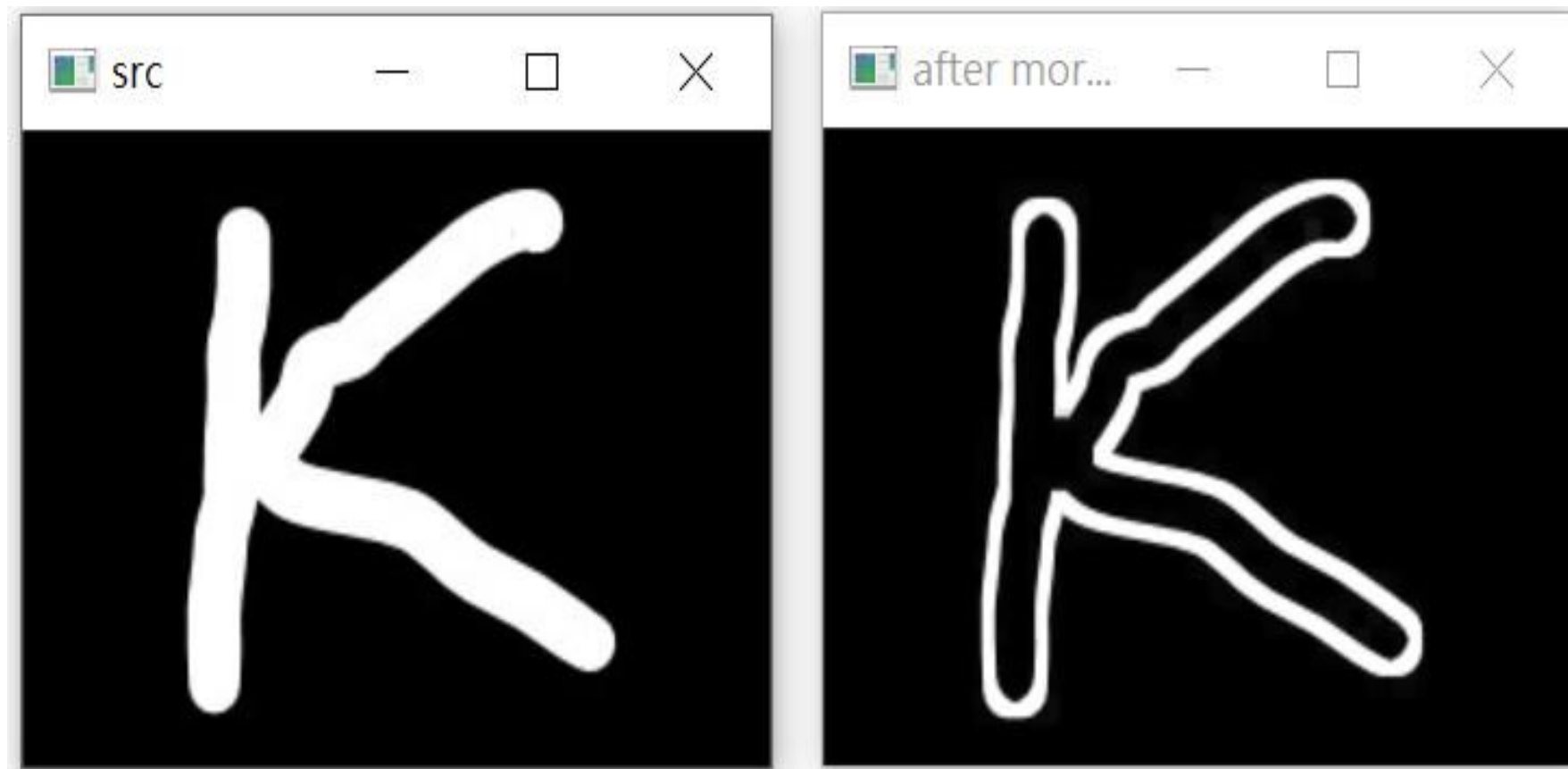


膨脹影像 - 腐蝕影像

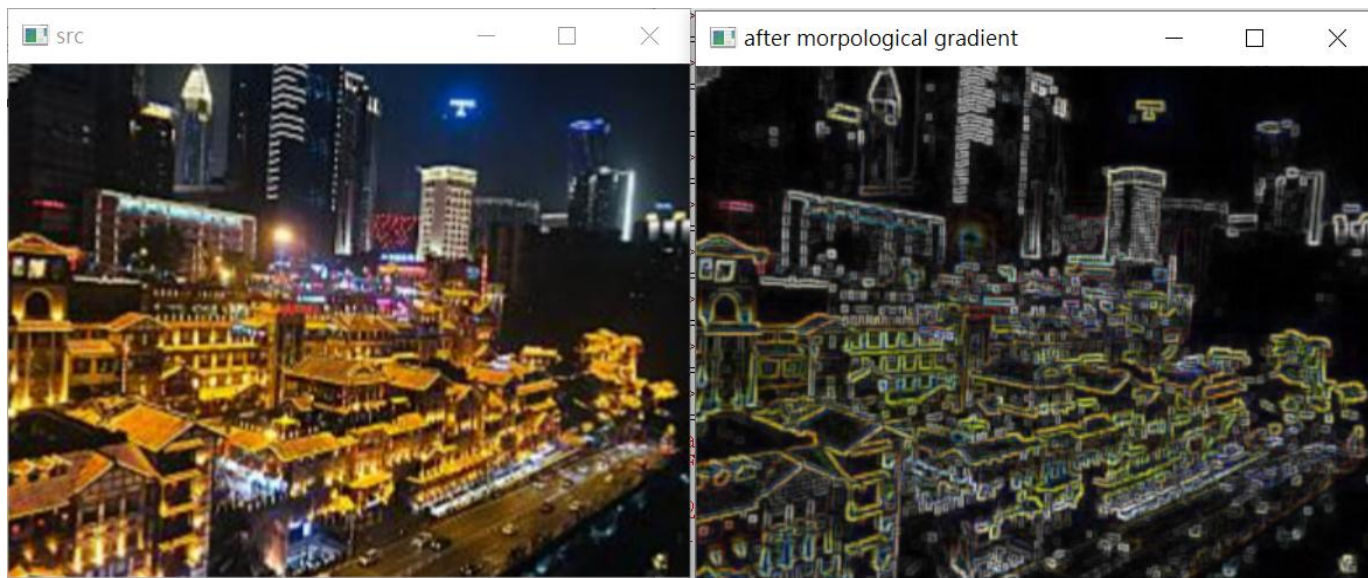
- 程式實例ch12_15.py：先處理k.jpg影像的膨脹與腐蝕。



- 程式實例16.py：使用形態學梯度獲得k.jpg影像邊緣的設計。

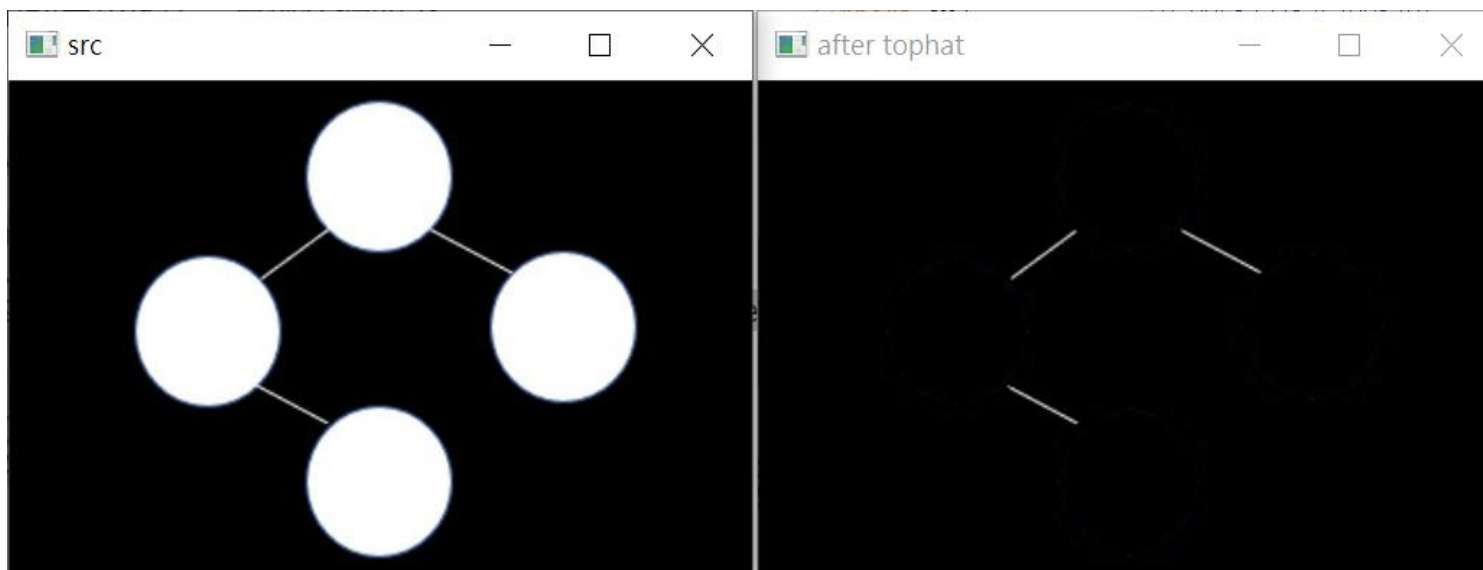


- 程式練習：讀取hole.jpg，內核設為 3×3 ，將影像改為hole.jpg。



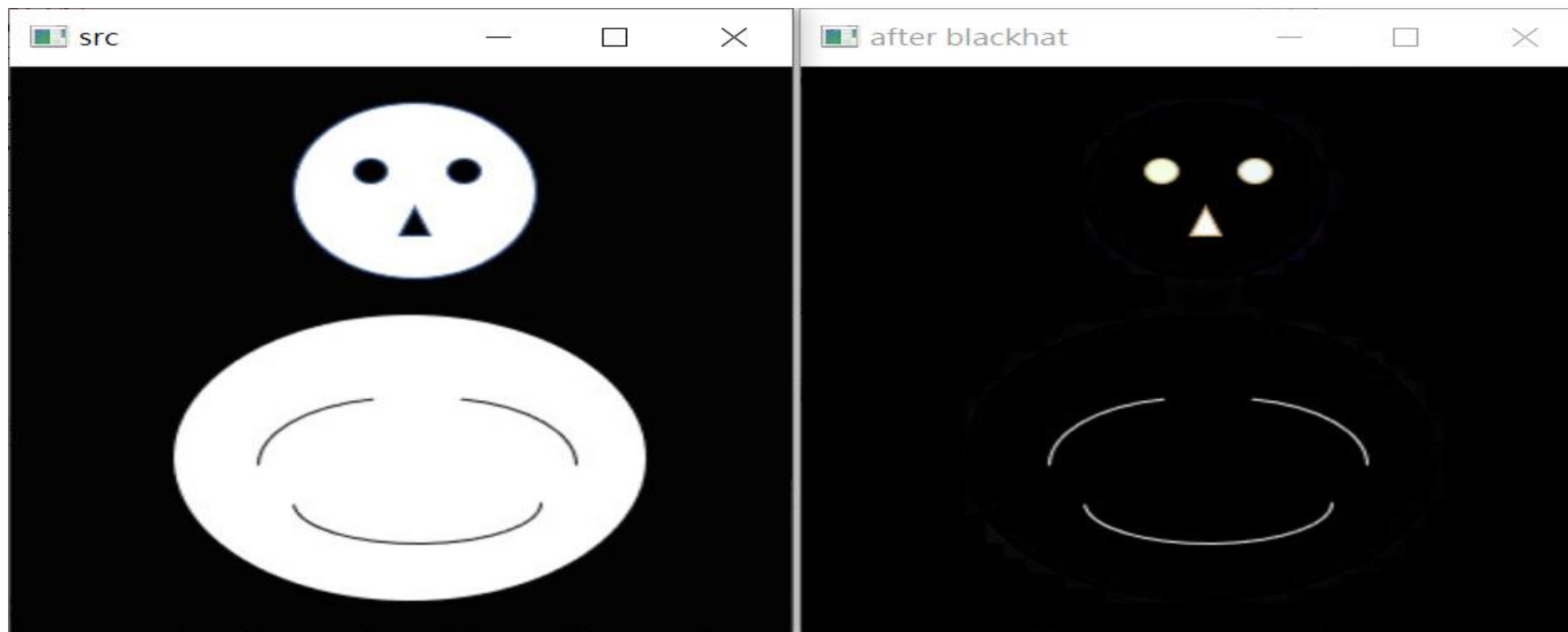
12-7：禮帽運算(tophat)

- 禮帽運算(tophat)觀念是將原始影像減去開運算的影像(先腐蝕再膨脹)，經過這個操作後可以得到原始影像的噪音訊息，或是得到比原影像的邊緣更亮的影像訊息。
- 程式實例：使用禮帽運算獲得噪音訊息的應用。



12-8：黑帽運算(blackhat)

- 黑帽運算(blackhat)觀念是將原始影像減去閉運算的影像，經過這個操作後可以得到原始影像內部的細節，或是得到比原影像的邊緣更黑暗の影像訊息。
- 程式實例：使用黑帽運算獲得內部細節の應用。



- 程式實例：使用excel.jpg影像檔案進行黑帽運算。

```
import cv2
import numpy as np

src = cv2.imread("excel.jpg")
kernel = np.ones((11,11),np.uint8)      # 建立11x11內核
dst = cv2.morphologyEx(src,cv2.MORPH_BLACKHAT,kernel)  # blackhat

cv2.imshow("src",src)
cv2.imshow("after blackhat",dst)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

